Midterm Report: Connect Four, Train on 8x7, Test on 9x8

AI | Group 3 | TJ Raklovits, Andrew Roddy

Topic

We will make AI agents for Connect Four on boards that are larger than the standard 7x6 board. Many published studies use 7x6. We will train the agents on an 8x7 board (8 columns, 7 rows), then we will use a 9x8 board for the test.

Problem

John Tromp solved 8x7 to determine that with perfect play, the first player wins. As a result, an exact solver can give the correct result for each 8x7 position. No published result exists for 9x8.

We will not try to solve 9x8. Our question is: can agents that we train on 8x7 play well on 9x8? We also want to know why each method works or fails. TJ Raklovits will make the solver and three agents. Andrew Roddy will make four agents. All agents do the same task: they select a column for a given board.

Dataset and Environment

- **Training data:** 8x7 positions with labels from an exact solver. We will write this solver. It will use alpha-beta search on bitboards with a transposition table, like published Connect Four solvers.
  - The positions will come from games with random moves that do not lose at once, with no duplicates or mirror images. Each position has 56 cells. Its label gives the result (win, draw, or loss) of each legal column.
  - Target size: about 100,000 positions from the middle and the end of the game. A separate set of about 2,000 opening positions is only for the test.
- **Solver check:** the UCI Connect-4 dataset (John Tromp, UCI Machine Learning Repository) has 67,557 7x6 positions with win, draw, or loss labels. Pascal Pons published 6,000 more 7x6 test positions. The solver must give the same results for all of them before we use it.
- **Test board:** 9x8. This board has no labels. The agents play games against each other from random opening positions.
- **Environment:** we will write as much as possible in Rust. This includes the game engine for both board sizes, the solver, all agents, the match program, and the tournament. The neural network will use a Rust deep learning library (for example Burn).

Proposed Agents (can change)

Member	Agent	Key question
TJ Raklovits	Exact solver as an agent, with an opening book	How strong is it on 9x8 with a time limit?
TJ Raklovits	Alpha-beta, handcrafted evaluation (threats, row parity)	How deep must it search? Does it work on 9x8?
TJ Raklovits	Alpha-beta, logistic-regression evaluation from 8x7 data	Is the learned evaluation better on 9x8?
TJ Raklovits	Rule agent with the strategy rules of Victor Allis, no search	Do the strategy rules work on 9x8?
Andrew Roddy	Monte Carlo Tree Search (UCT)	How many simulations does it need?
Andrew Roddy	Fully convolutional network, trained on 8x7 labels	Does its knowledge transfer to 9x8?
Andrew Roddy	MCTS with the network as a guide (like AlphaZero)	Is it better than search alone?
Andrew Roddy	Safe random baseline: random, but takes wins and blocks threats	It is a low bar and the Elo anchor.

Evaluation

- **8x7 (training board):** we measure move accuracy against a solver. We use positions that we keep out of training, and the opening set. We report the accuracy for each phase of the game. The agents also play matches against the solver.
- **9x8 (test board): agent tournament.** Each agent plays each other agent, with a fixed time for each move. Each pair plays from the same opening positions, once as each side.
  - We rank the agents by win rate and Elo.

Limitations

- We cannot find perfect play on 9x8 with our compute. As a result, 9x8 has no move accuracy and no perfect opponent. The tournament gives only relative strength.
- It is possible that the solver can solve late 9x8 positions. Then exact checks are possible, but only near the end of the game.
- Some parts depend on the 8x7 board: the input size, the learned weights, and the opening book. We must change these parts for 9x8. The neural network will be fully convolutional, so that it can use a 9x8 board.